Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЛАБОРАТОРНА РОБОТА №3

з дисципліни «Об’єктно-орієнтоване програмування»

на тему

«Розробка інтерфейсу користувача на C++»

Виконав:
Чакун Олександр Сергійович
студент групи ІМ-о51
номер у списку: 16

Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

1. Мета роботи

Отримати практичні навички створення графічного інтерфейсу користувача на C++ для Windows із використанням інкапсуляції, успадкування, поліморфізму, меню, панелі інструментів Toolbar та підказок для кнопок.

2. Завдання

- Створити Win32-проєкт Lab3 у Microsoft Visual Studio C++.
- Продовжити графічний редактор попередньої лабораторної роботи.
- Додати Toolbar з кнопками, що дублюють пункти меню «Об’єкти».
- Реалізувати tooltips для кнопок Toolbar.
- Обробники повідомлень, зокрема OnNotify, реалізувати як методи класу.
- Перевірити роботу програми, проаналізувати код та оформити звіт.

3. Розрахунок варіанта

Для лабораторної роботи №3 використовується значення Ж = Жлаб2 + 1. У лабораторній роботі №2 номер студента у списку був 16, тому для Lab3 маємо Ж = 17.

```
Ж = 16 + 1 = 17
N = Ж + 100 = 117
17 mod 3 = 2
17 mod 4 = 1
17 mod 2 = 1
17 mod 5 = 2
17 mod 6 = 5
```

- масив вказівників Shape* - статичний, на 117 елементів;
- «гумовий» слід - суцільний червоний;
- прямокутник вводиться від центра до одного з кутів і має сіре заповнення;
- еліпс вводиться за двома протилежними кутами та відображається без заповнення;
- активний режим редагування показується в заголовку головного вікна;
- Toolbar містить 4 кнопки: «Крапка», «Лінія», «Прямокутник», «Еліпс», кожна має tooltip.

4. Структура програми

```
Lab3.cpp
├── shape_editor.h / shape_editor.cpp
│   ├── editor.h / editor.cpp
│   ├── shape.h / shape.cpp
│   └── resource.h
└── Lab3.rc
    ├── resource.h
    └── toolbar.bmp
```

Головний файл Lab3.cpp приймає повідомлення Windows і передає їх об’єкту ShapeObjectsEditor. Класи Shape та Editor задають абстрактні інтерфейси, а похідні класи реалізують поведінку конкретних фігур і режимів редагування.

5. Діаграма класів

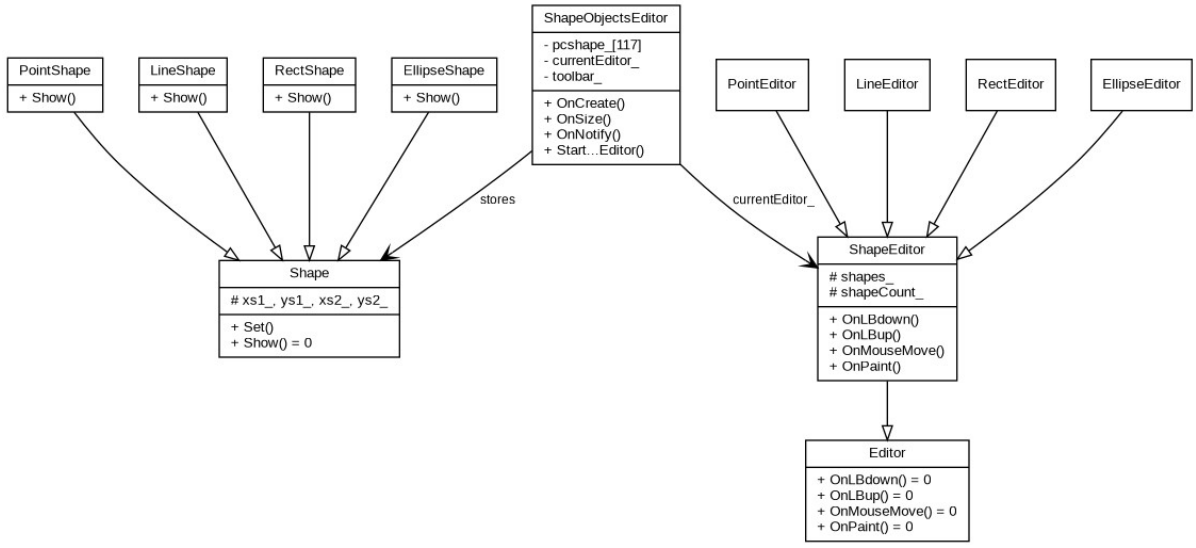


Рис. 1. Діаграма класів Lab3

6. Основні фрагменти вихідного коду

6.1. Обробка повідомлень у Lab3.cpp

Функція WndProc приймає повідомлення головного вікна. Повідомлення створення, зміни розміру, сповіщень Toolbar, миші та перемальовування передаються відповідним методам класу ShapeObjectsEditor.

```
case WM_CREATE:
    g_editor.OnCreate(hWnd);
    return 0;

case WM_SIZE:
    g_editor.OnSize(hWnd);
    return 0;

case WM_NOTIFY:
    g_editor.OnNotify(hWnd, wParam, lParam);
    return 0;

case WM_LBUTTONDOWN:
    g_editor.OnLBdown(hWnd);
    return 0;

case WM_LBUTTONUP:
    g_editor.OnLBup(hWnd);
    return 0;

case WM_MOUSEMOVE:
    g_editor.OnMouseMove(hWnd);
    return 0;

case WM_PAINT:
    g_editor.OnPaint(hWnd);
    return 0;
```

6.2. Вибір режиму через меню та Toolbar

```
case IDM_POINT:
    g_editor.StartPointEditor(hWnd);
    return 0;

case IDM_LINE:
    g_editor.StartLineEditor(hWnd);
    return 0;

case IDM_RECT:
    g_editor.StartRectEditor(hWnd);
    return 0;

case IDM_ELLIPSE:
    g_editor.StartEllipseEditor(hWnd);
    return 0;
```

Пункти меню та кнопки Toolbar використовують однакові ідентифікатори команд, тому вони запускають ті самі методи редактора.

6.3. Абстрактний клас Shape

```
class Shape
{
protected:
   long xs1_{};
   long ys1_{};
   long xs2_{};
   long ys2_{};

public:
   virtual ~Shape() = default;

   void Set(long x1, long y1, long x2, long y2);
   virtual void Show(HDC hdc) const = 0;
};
```

Метод Show є чисто віртуальним. Тому клас Shape є абстрактним, а конкретне відображення задається у PointShape, LineShape, RectShape та EllipseShape.

6.4. Абстрактний клас Editor

```
class Editor
{
public:
   virtual ~Editor() = default;
   virtual void OnLBdown(HWND hWnd) = 0;
   virtual void OnLBup(HWND hWnd) = 0;
   virtual void OnMouseMove(HWND hWnd) = 0;
   virtual void OnPaint(HWND hWnd) = 0;
};
```

Клас Editor визначає спільний інтерфейс для обробки подій миші та перемальовування. Конкретні редактори перевизначають ці методи відповідно до типу фігури.

6.5. Статичний масив для варіанта Ж = 17

```
static constexpr int kMaxShapes = 117;
Shape* pcshape_[kMaxShapes]{};
int shapeCount_{};
```

Оскільки 17 mod 3 = 2, використовується статичний масив. Кількість елементів N = 17 + 100 = 117.

6.6. Створення Toolbar

Toolbar створюється як дочірнє вікно головного вікна. У проєкті використовується власний bitmap-ресурс toolbar.bmp з чотирма зображеннями кнопок.

```
toolbar_ = CreateWindowExW(
    0,
    TOOLBARCLASSNAMEW,
    nullptr,
    WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT |
        TBSTYLE_TOOLTIPS | CCS_TOP,
    0, 0, 0, 0,
    hWnd,
    reinterpret_cast<HMENU>(
        static_cast<INT_PTR>(IDC_MAIN_TOOLBAR)),
    GetModuleHandleW(nullptr),
    nullptr);
```

6.7. Підказки для кнопок

```
void ShapeObjectsEditor::OnNotify(HWND, WPARAM, LPARAM lParam)
{
    const LPNMHDR header = reinterpret_cast<LPNMHDR>(lParam);

    if (header == nullptr || header->code != TTN_GETDISPINFOW)
        return;

    auto* info = reinterpret_cast<NMTTDISPINFOW*>(lParam);

    switch (static_cast<UINT>(info->hdr.idFrom))
    {
    case IDM_POINT:
        info->lpszText = const_cast<LPWSTR>(L"Крапка");
        break;
    case IDM_LINE:
        info->lpszText = const_cast<LPWSTR>(L"Лінія");
        break;
    case IDM_RECT:
        info->lpszText = const_cast<LPWSTR>(L"Прямокутник");
        break;
    case IDM_ELLIPSE:
        info->lpszText = const_cast<LPWSTR>(L"Еліпс");
        break;
    }
}
```

6.8. Активний режим у заголовку

```
void ShapeObjectsEditor::SetModeTitle(
    HWND hWnd,
    const wchar_t* modeName)
{
    wchar_t title[160]{};
    wsprintfW(
```

```
        title,
        L"Лабораторна робота №3 - Чакун Олександр Сергійович - Режим: %s",
        modeName);

    SetWindowTextW(hWnd, title);
}
```

7. Результати роботи програми

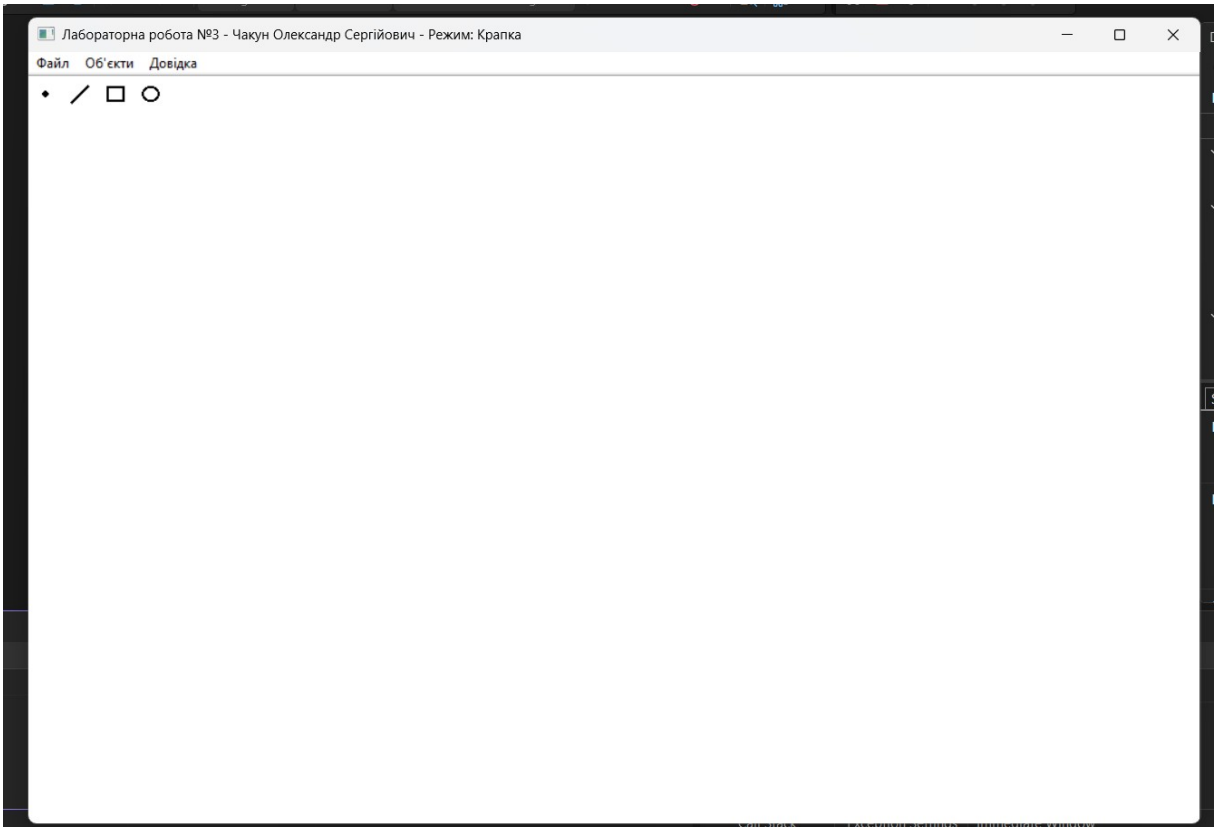


Рис. 2. Головне вікно програми з Toolbar

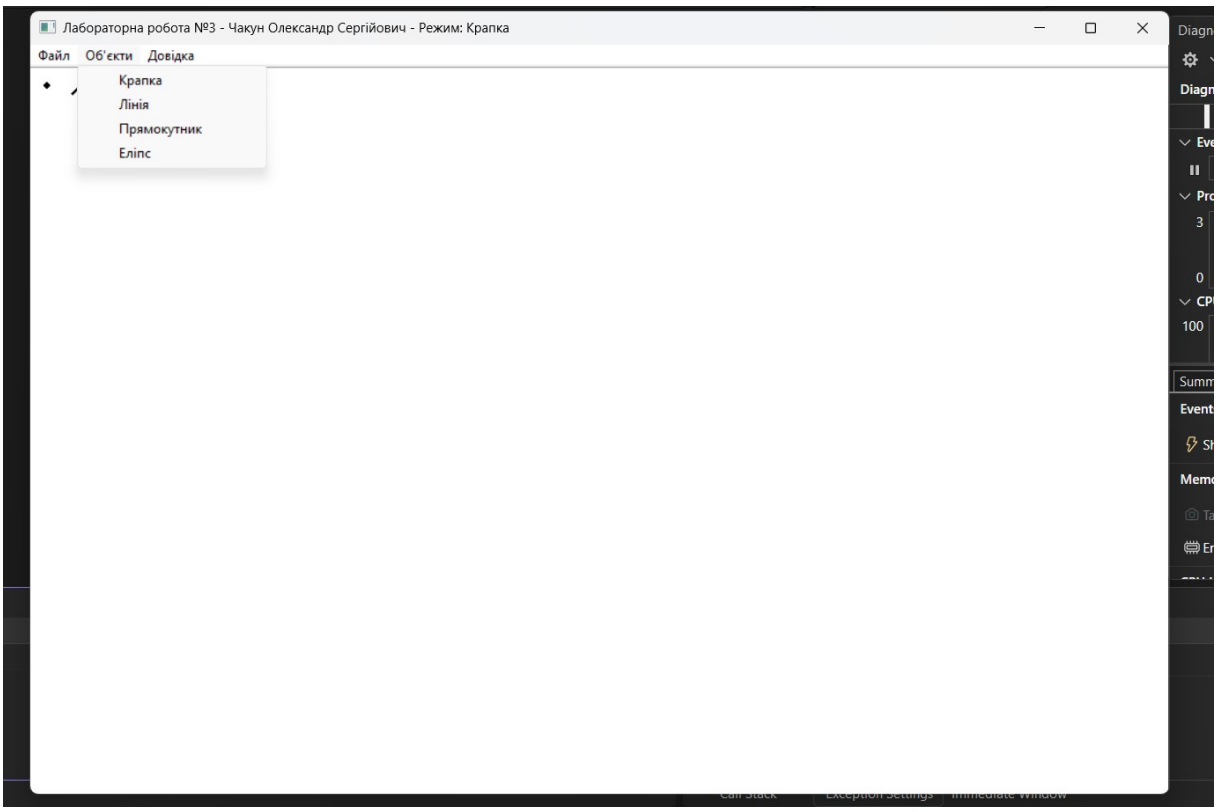


Рис. 3. Меню «Об’єкти»

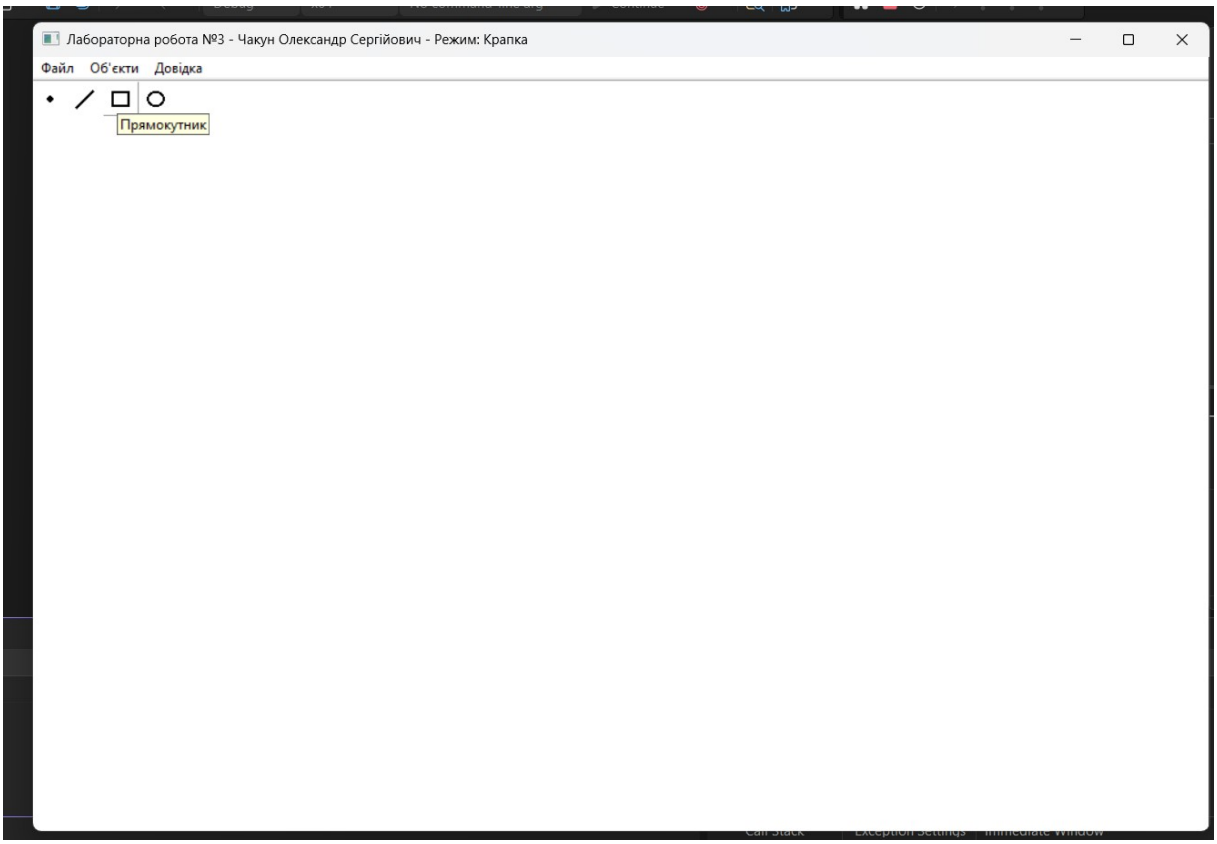


Рис. 4. Підказка (tooltip) для кнопки Toolbar



Рис. 5. Відображення активного режиму в заголовку вікна

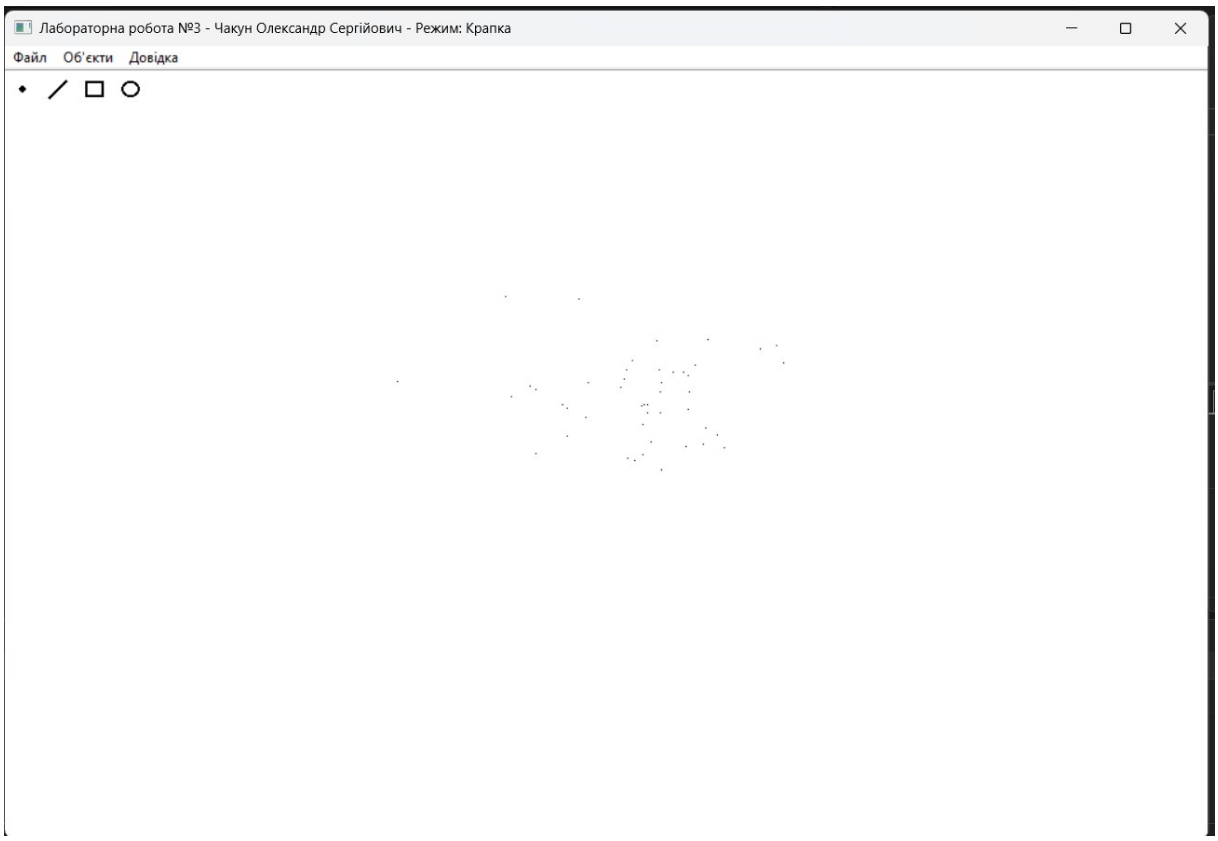


Рис. 6. Режим введення точок

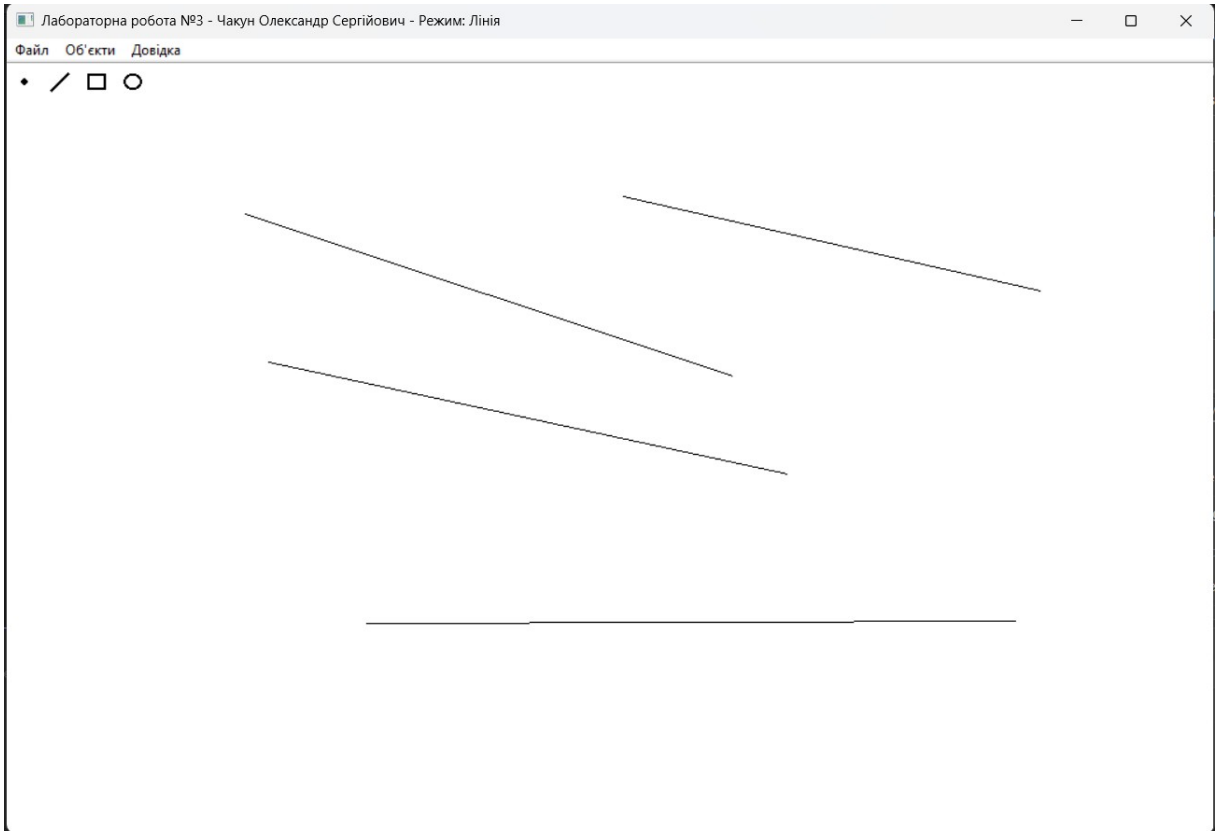


Рис. 7. Режим введення ліній

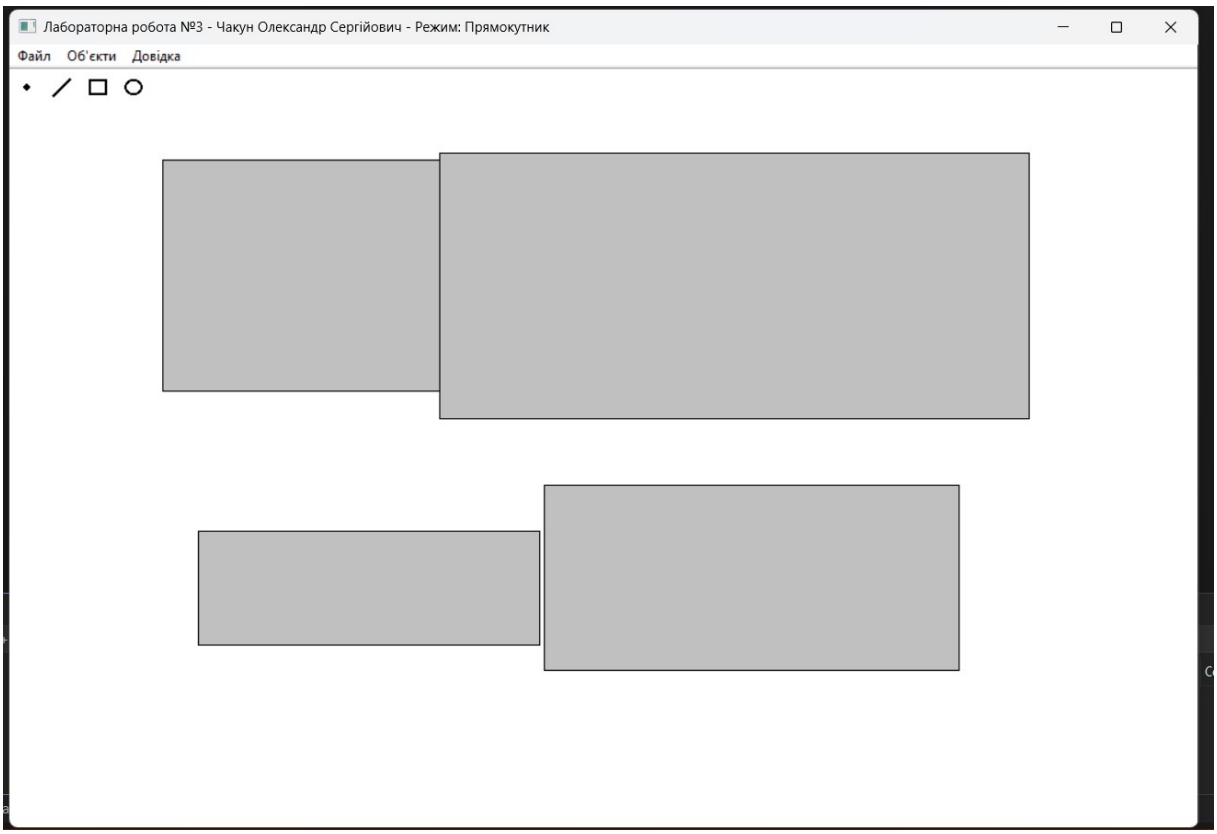


Рис. 8. Прямокутники для варіанта Ж = 17: чорний контур та сіре заповнення

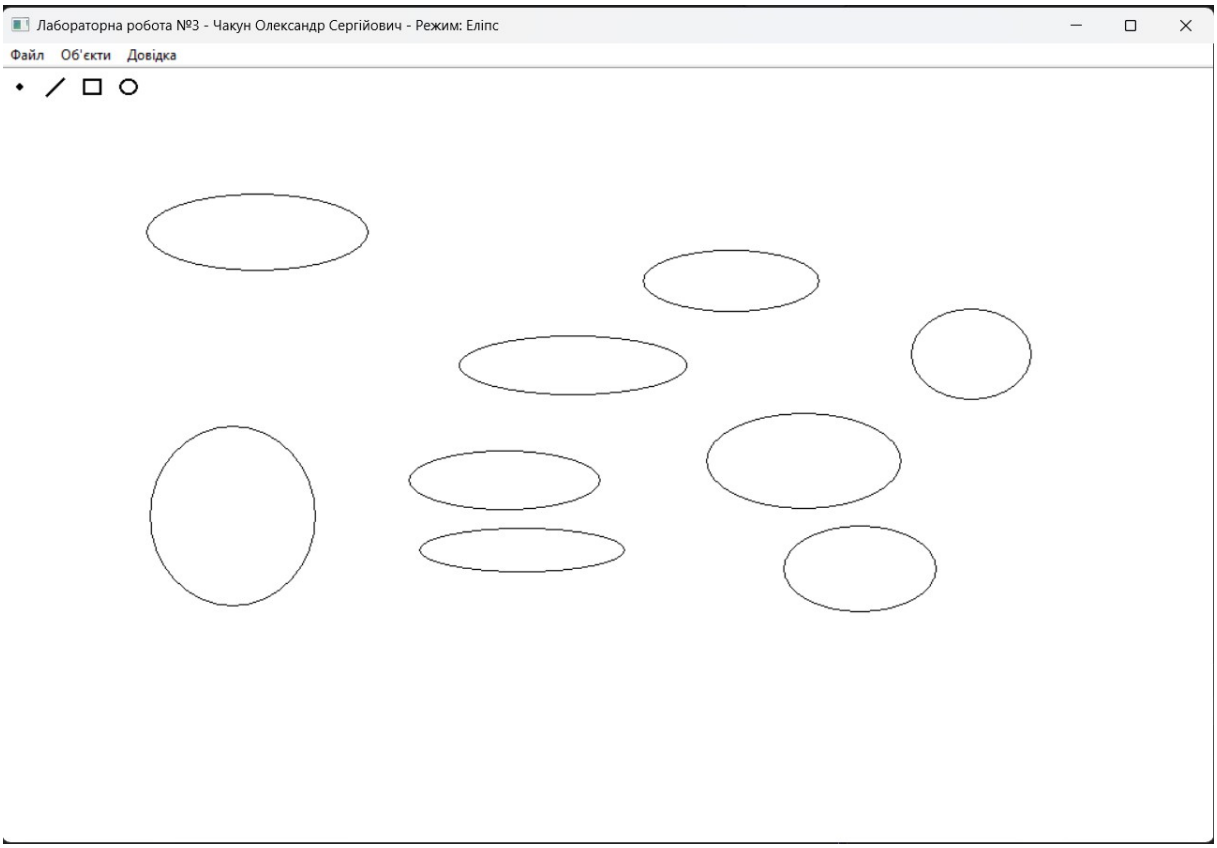


Рис. 9. Еліпси для варіанта Ж = 17: контур без заповнення

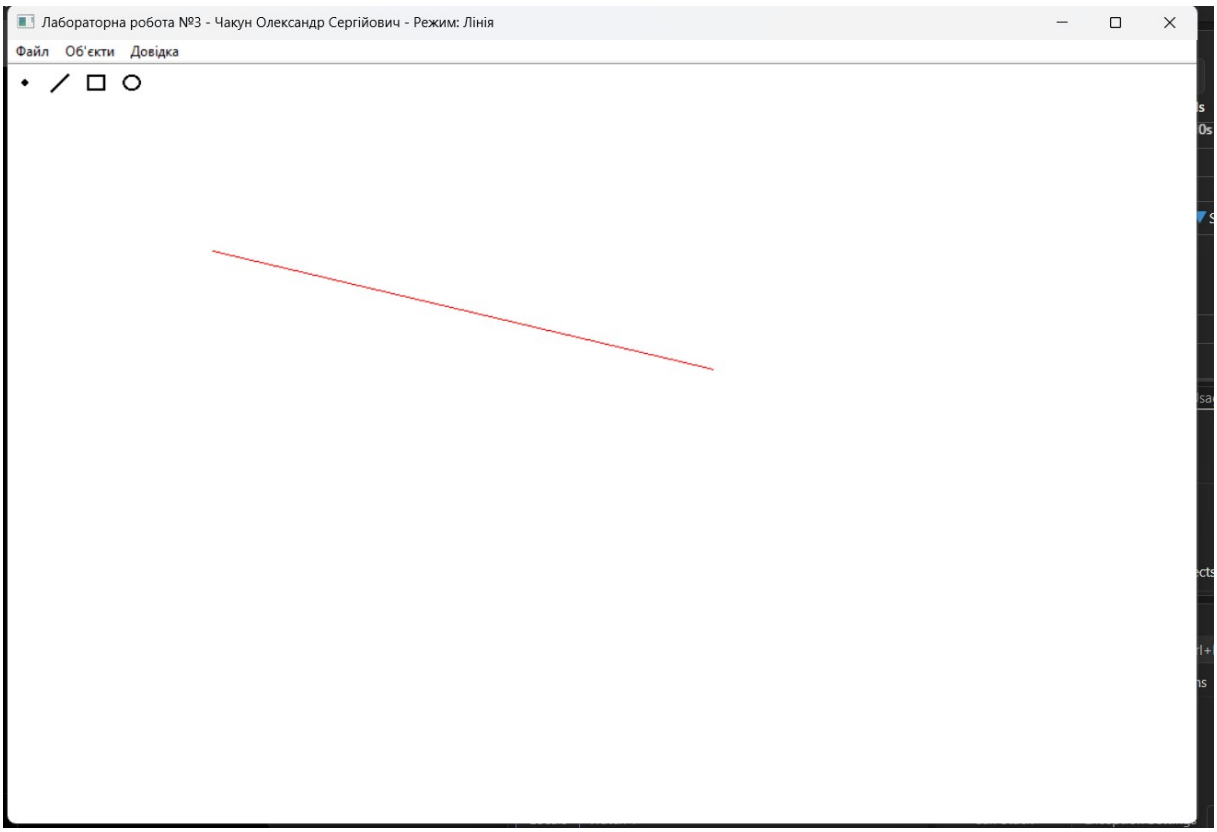


Рис. 10. Суцільний червоний «гумовий» слід під час введення об’єкта



Рис. 11. Вибір режиму за допомогою Toolbar

8. Висновки

У лабораторній роботі №3 графічний редактор було доповнено повноцінним користувацьким інтерфейсом із Toolbar. Кнопки панелі дублюють команди меню «Об’єкти», мають власні зображення та текстові підказки. Для обробки підказок використано повідомлення WM_NOTIFY, а для адаптації панелі до розміру вікна - WM_SIZE.

Збережено об’єктно-орієнтовану структуру програми: абстрактні класи Shape та Editor задають спільні інтерфейси, а похідні класи реалізують поліморфне відображення фігур та їх редагування. Для варіанта Ж = 17 реалізовано статичний масив на 117 вказівників Shape, червоний суцільний «гумовий» слід, прямокутник із сірим заповненням, контурний еліпс та показ активного режиму в заголовку головного вікна.

Роботу програми перевірено на практиці: усі чотири режими введення фігур, меню, Toolbar, tooltips та індикація активного режиму працюють коректно.